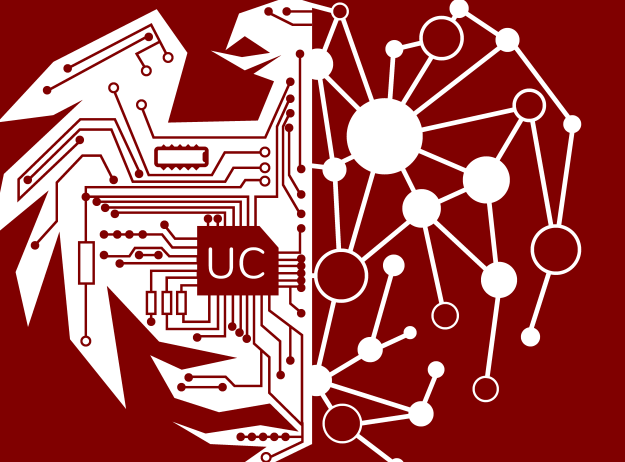




KVMSR+UDWeave: Extreme-Scaling with Fine-grained Parallelism on the UpDown Graph Supercomputer



Alexander Fell¹, Yuqing Wang¹, Tianshuo Su¹, Marziyeh Nourian¹, Wenyi Wang¹,
Jose M. Monsalve-Diaz¹, Andronicus Rajasukumar¹, Jiya Su¹, Ruiqi Xu¹, Rajat Khandelwal¹, Tianchi Zhang¹, David F. Gleich³, Yanjing Li¹, Hank Hoffmann¹, Andrew A. Chien^{1 2}

¹University of Chicago ²Argonne National Laboratory ³Purdue University

Why Graph Processing Is Hard

High-performance computing (HPC) has achieved efficient, scalable application performance for standard problems using MPI+X. However, there are large classes of problems with irregular parallel computation and data structures that remain unconquered.

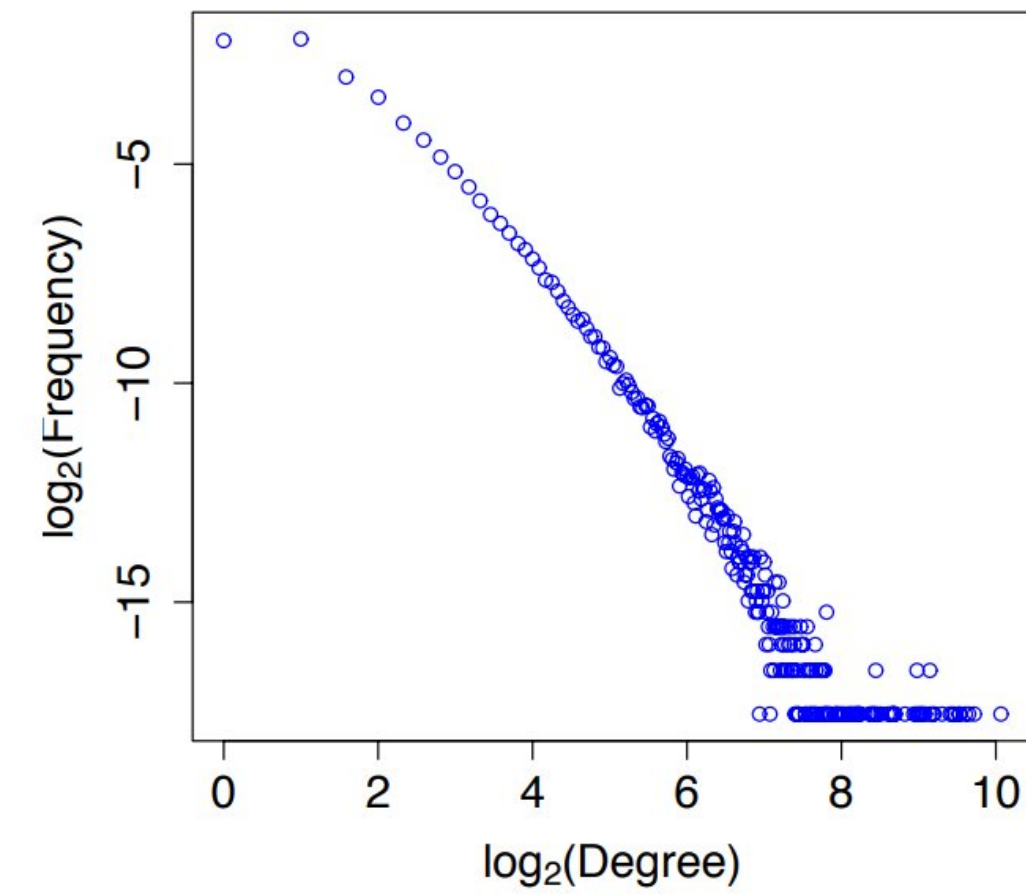


Figure 1. Graph computing has per-vertex/pre-edge parallelism with a high skew.

- Real-time Graph Processing poses extreme performance demands
 - Petabytes of Graph data
 - Billions of fine-grained tasks (vertex/edge)
 - Limited data reuse
- Skewed power-law degree distribution: extreme irregularity in data and computation
 - Unbalanced compute & poor data locality

Three Dimensions of a Parallel Program

- Parallel Computation:** Express and manage parallelism
- Computation Binding:** Bin computation to parallel compute elements
- Data Layout:** Organization of the data in the memory

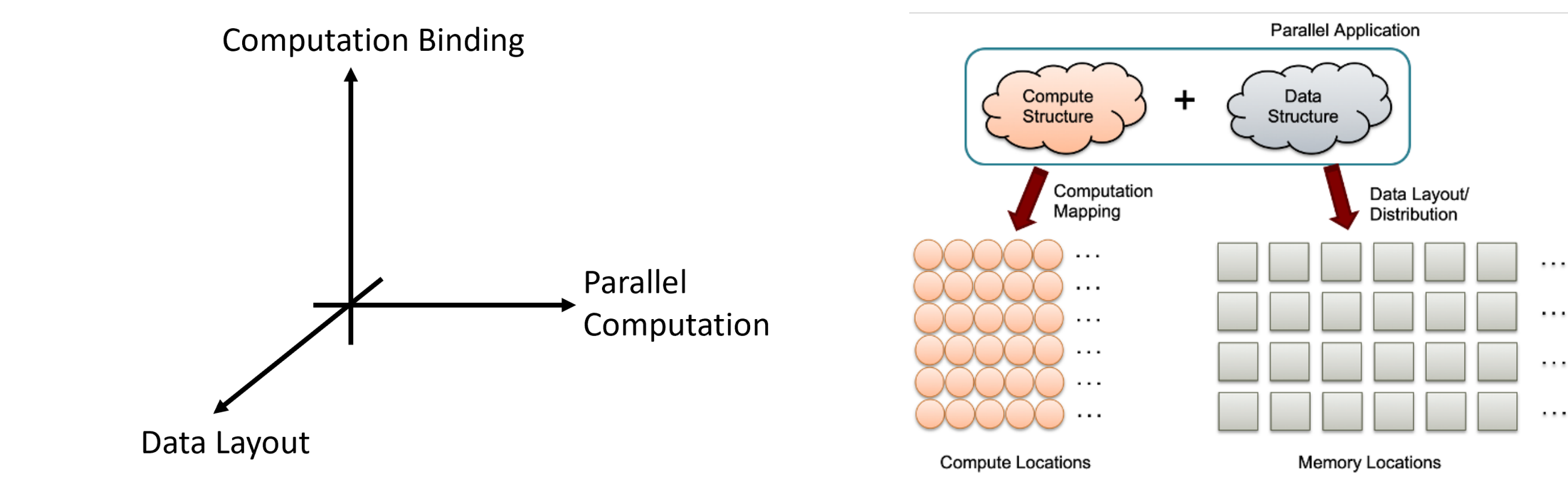
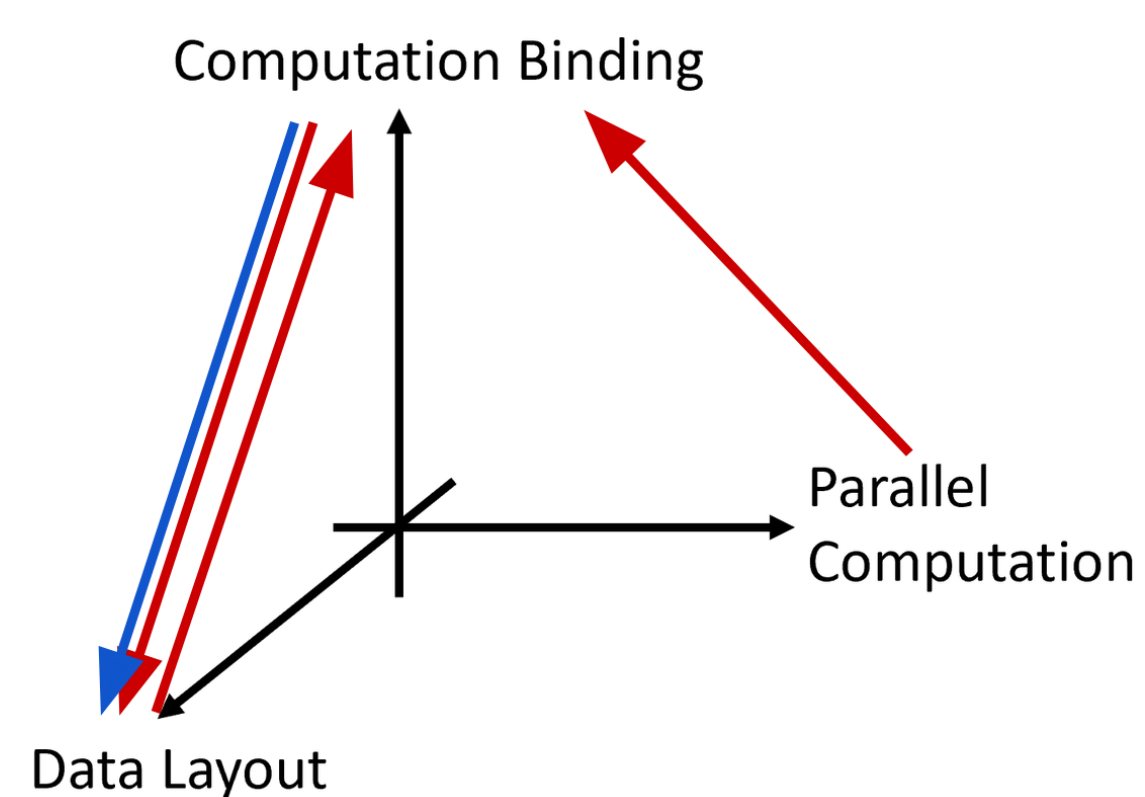


Figure 2. 3 Dimensions of a Parallel Program

Existing Parallel Programming Models tie Dimensions together!



For irregular applications, coupling makes load balance and data distribution difficult.

MPI: Computation Binding, Data Placement, and Parallel Computation depend on each other. E.g., the location of the computation depends on the machine on which the data resides. If the data layout changes, so does the computation binding.

PGAS: Computation Binding depends on Data Placement.

UpDown Graph Supercomputer

Each UpDown Graph Supercomputer consists of 16,384 UpDown Nodes, each equipped with 2,048 compute lanes, designed for efficient fine-grained parallelism. Each lane operates independently and includes mechanisms to respond to hardware asynchrony and variable latency efficiently.

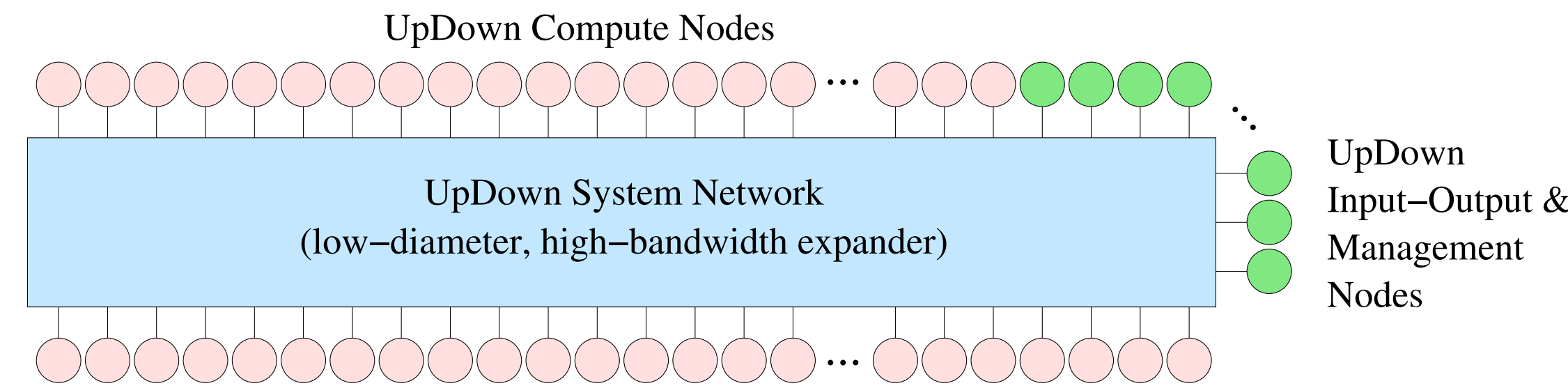
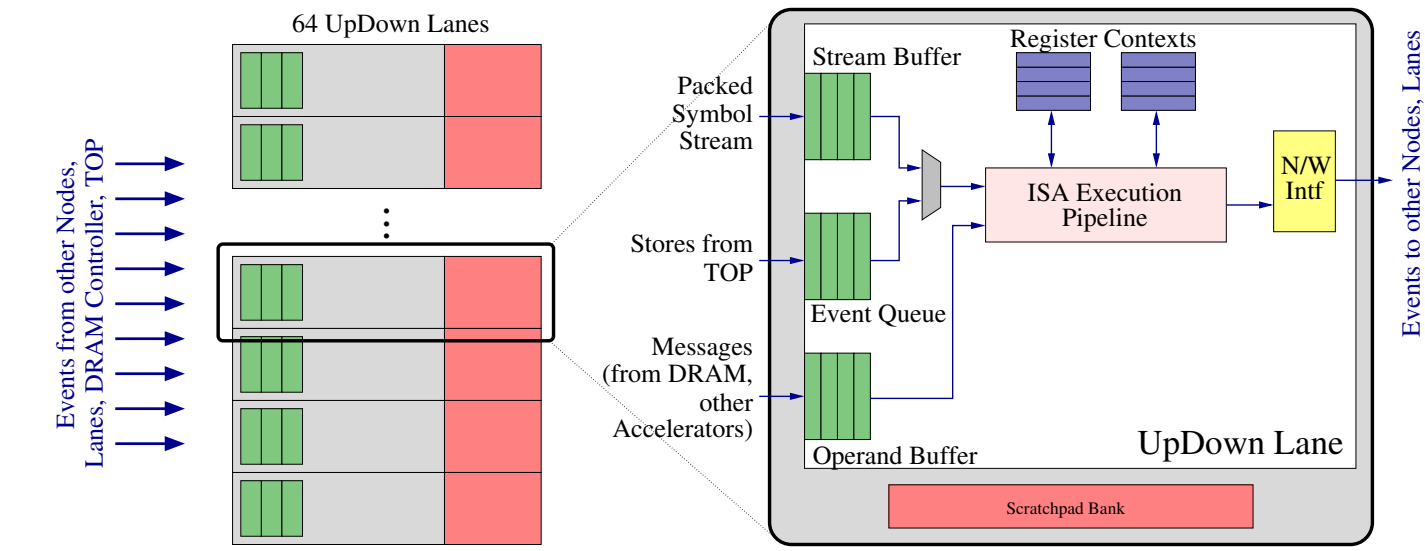


Figure 3. 16k UpDown Nodes are connected by a scalable Polarstar network.

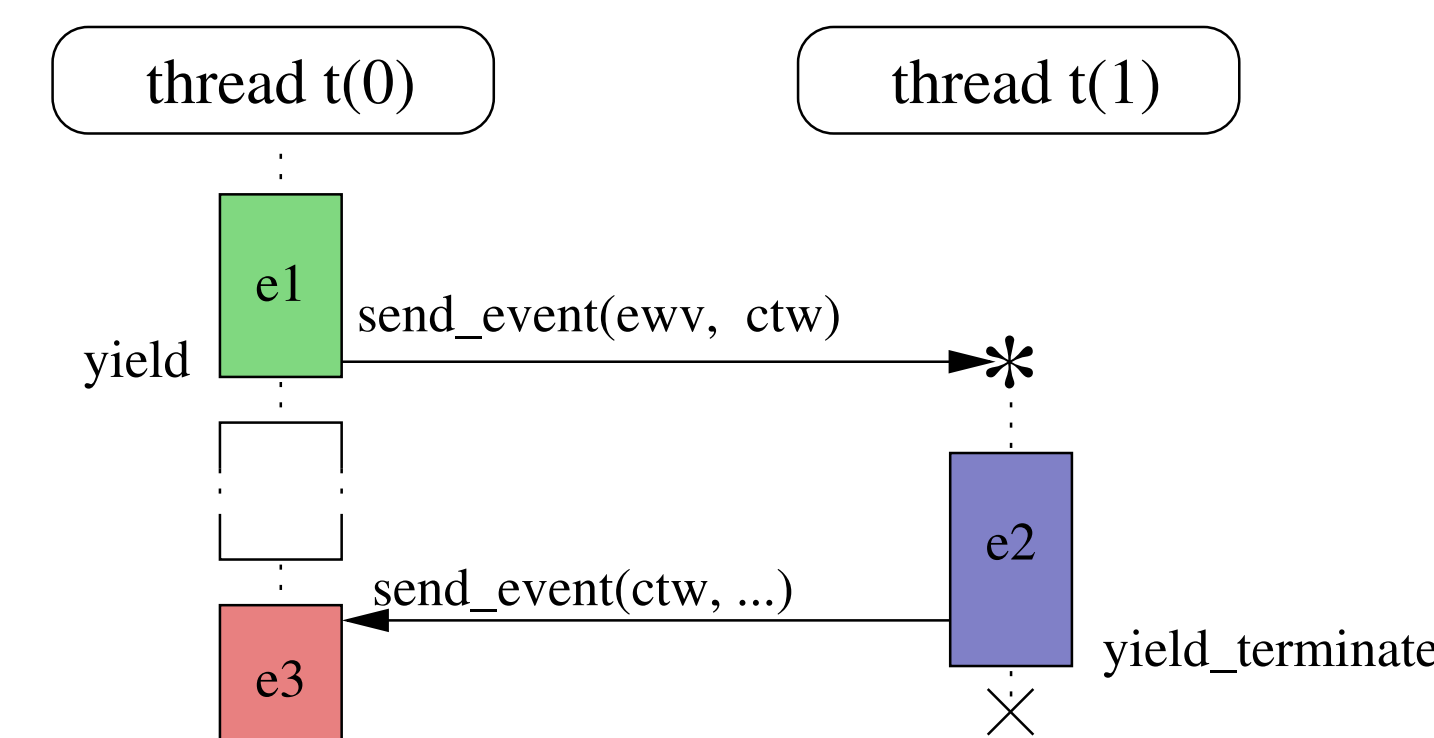
Fine-grained parallelism is supported by UpDown Lanes, with 1-cycle, event-driven scheduling, 128-way multithreading, and ISA split-transaction DRAM access. 10-instruction thread invocations are efficient.

- Millions of multi-threaded processing units (UpDown Lanes)
- Event-driven Tasks
- Short threads with 1 cycle activation, switching, and termination
- Fast and direct messaging between Lanes
- Global address space
- Petabytes of Memory



Programming UpDown Lanes: UDWeave

```
1 thread TCallReturn {
2   event e1() {
3     print("I am in e1");
4     uint64_t evw = evw_new(curNetworkID+1, e2);
5     uint64_t ctW = evw_update_event(CEVNT, e3);
6     // send uint64 0 and uint64 1 to e2 on a new thread
7     send_event(evw, 0, 1, ctW);
8     yield;
9   }
10
11   event e2(uint64_t data0, uint64_t data1) {
12     print("I am in e2 and received this "
13           "data: %lu, %lu", data0, data1);
14     send_event(CCONT, IGNRCONT);
15     yield_terminate;
16   }
17
18   event e3() {
19     print("I am back from e2");
20     //...
21   }
22 }
```



Dimension 1: Parallel Computation (KVMSR)

```
1 event kv_map(Key key, uint64_t value0, uint64_t value1, double *value3) {
2   // source for the mapping function (parallel)
3   kv_map_emit(intermediate_key, intermediate_value);
4 }
5 event kv_reduce(Key intermediate_key, uint64_t intermediate_value) {
6   // source for the reduce function (parallel)
7   kv_reduce_emit(output_key, output_value);
8 }
```

Dimension 2: Computation Binding (KVMSR)

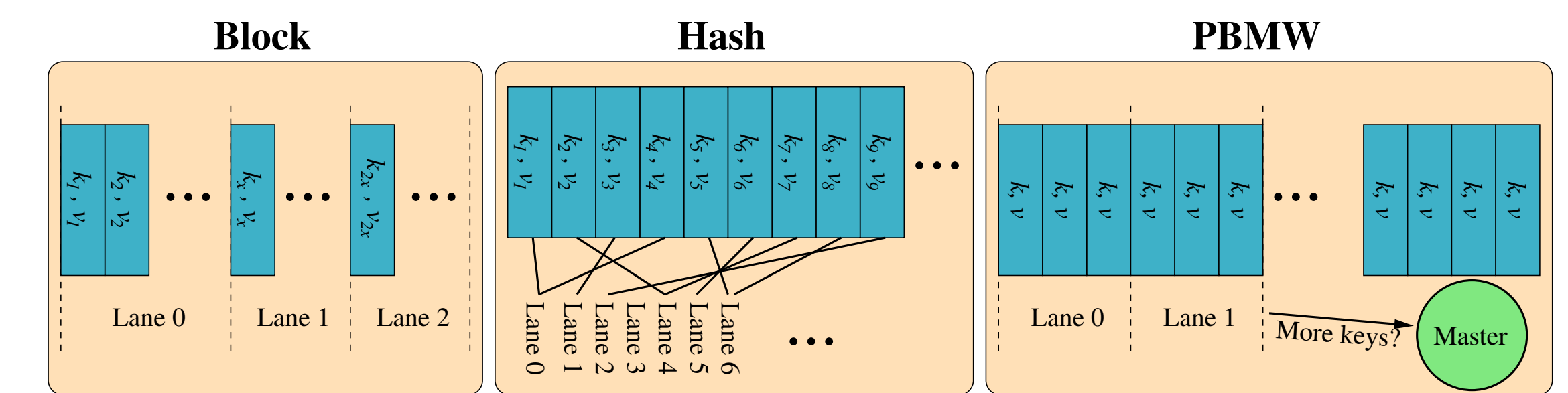
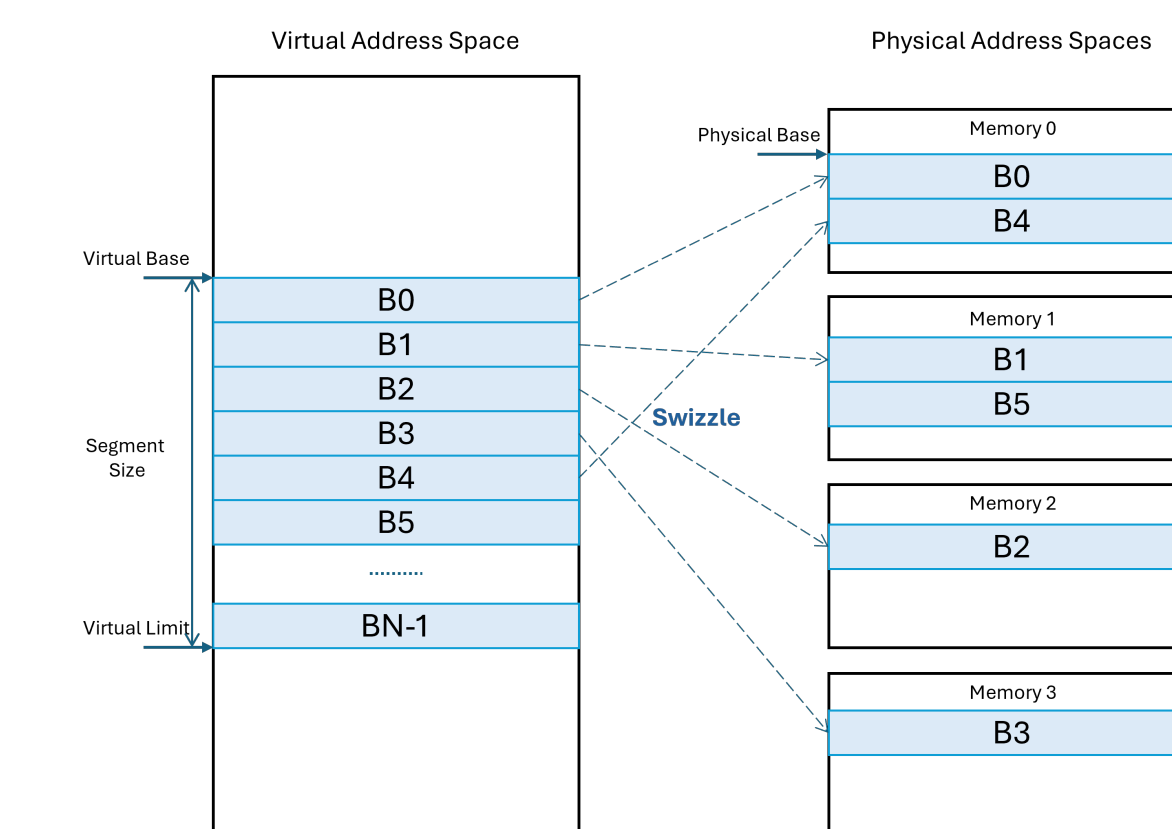


Figure 4. KVMSR creates tasks for each key, and can be used to express computation binding in a large-scale parallel UpDown program.

Dimension 3: Data Layout (DRAMalloc)

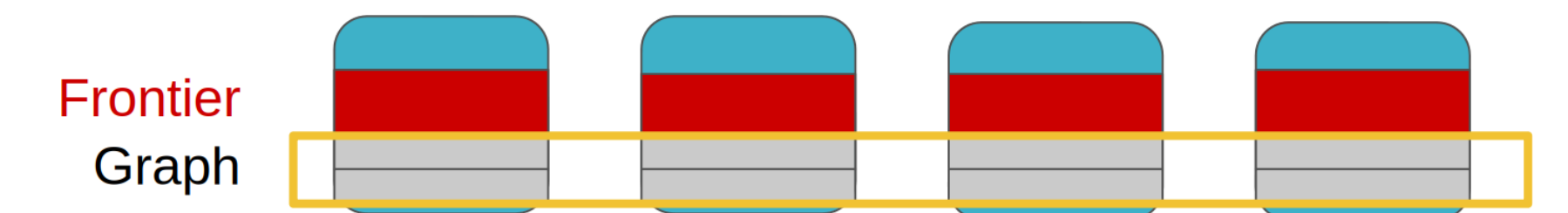
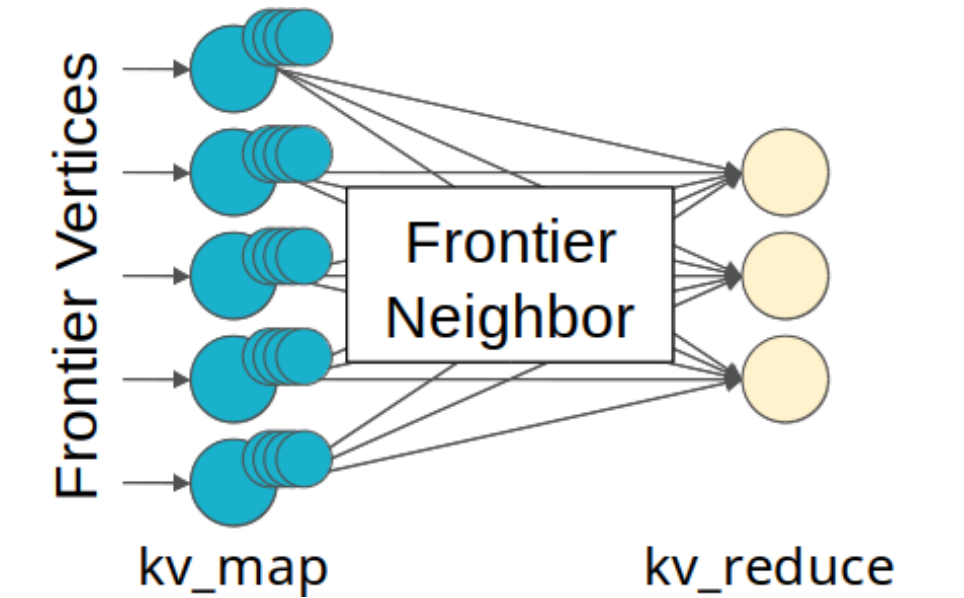


The user controls the layout for the global data structures. E.g., block size, start node, number of nodes. Examples

- (* , 0, 16k, 4096)
- (* , 0, 1k, 4096)
- (4TB, 4k, 8k, 1MB)

Example: Breadth-First Search

- Parallel Computation
 - kv_map:** compute BFS step on set of vertices
 - kv_reduce:** store next frontier
- Compute Binding
 - kv_map:** strided
 - kv_reduce:** hash
- Data Layout
 - Frontier: Node-local (large block)
 - Graph: Block-cyclic



We evaluate the full UpDown system's performance from 1 to 512 nodes (corresponding to 2,048 to 1,048,576 lanes) and include a few data points at 16,384 nodes.

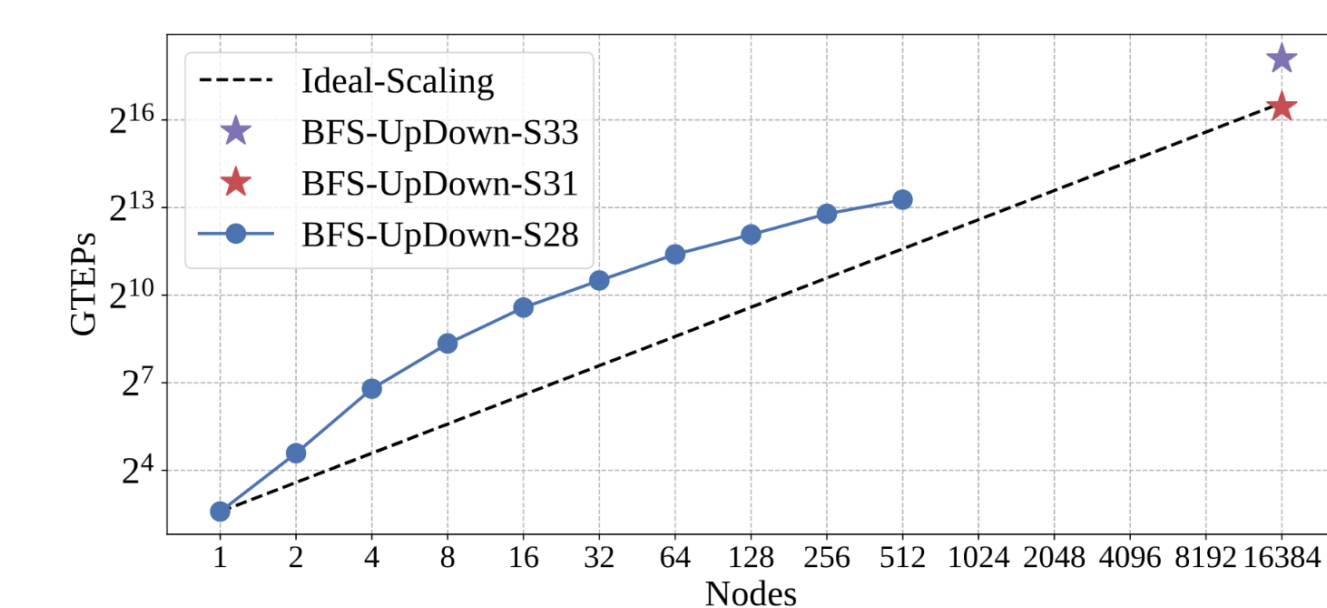


Figure 5. Giga traversed edges per second (GTEPS) for BFS on scale 28, 31, and 33 RMAT graphs

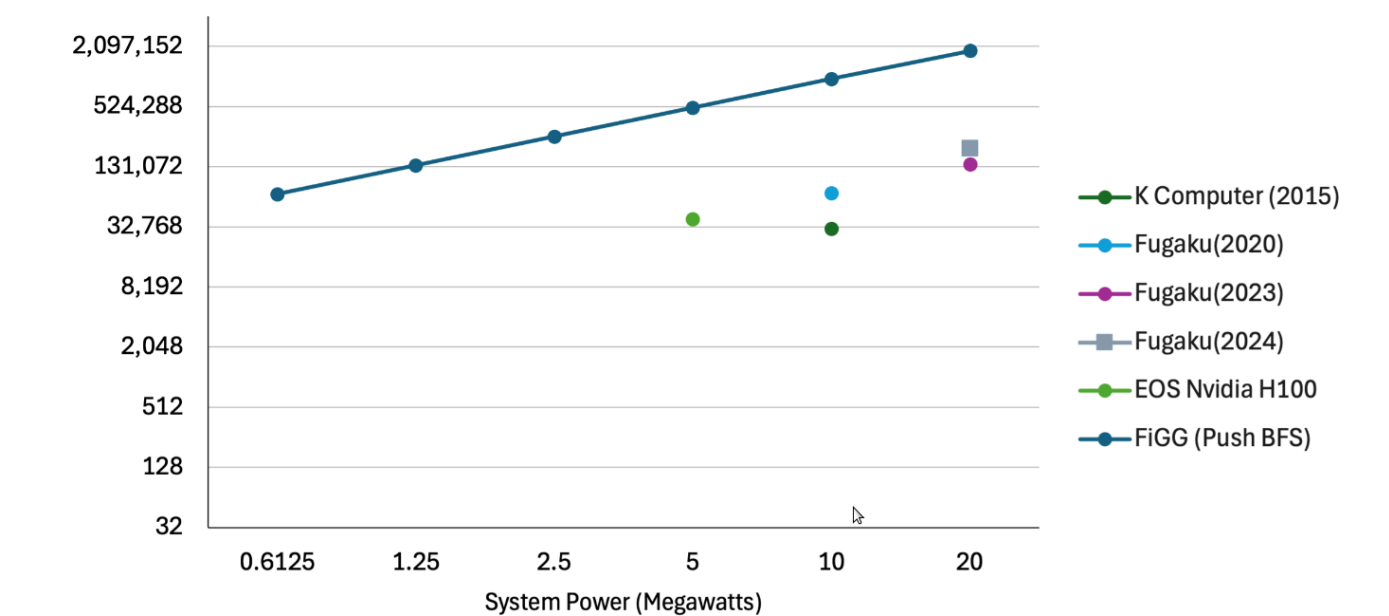


Figure 6. BFS Speedup: UpDown (Scale 40) compared to Graph 500 winners (2015, 2020, 2023, 2024) and the EOS GPU System.

References

Alexander Fell et. al., KVMSR+UDWeave: Extreme-Scaling with Fine-grained Parallelism on the UpDown Graph Supercomputer. In Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25). Association for Computing Machinery, New York, NY, USA, 1243–1262. <https://doi.org/10.1145/3731599.3767499>

